

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum- 590014, Karnataka.



LAB RECORD

on

Big Data Analytics (23CS6PCBDA)

Submitted by

THANMAY K H (1WA23CS018)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU - 560019

February 2025 – July 2025

B.M.S. College of Engineering

Bull Temple Road, Bangalore 560019

(Affiliated to Visvesvaraya Technological University, Belgaum)

Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “Big Data Analytics” carried out by **Thanmay K H(1WA23CS018)**, who is bonafide student of **B.M.S. College of Engineering**. It is in partial fulfilment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2026. The Lab report has been approved as it satisfies the academic requirements in respect of a Big Data Analytics (23CS6PCBDA) work prescribed for the said degree.

Leethavathi

Assistant Professor
Department of CSE, BMSCE

Dr. Kavitha Sooda

Professor & HOD
Department of CSE, BMSCE

INDEX

Sl. No.	Date	Experiment Title	Page No.
1	02.02.26	MongoDB- CRUD Operations Demonstration (Practice and Self Study)	1
2	09.02.26	Perform the following DB operations using Cassandra. • Create a keyspace by name Employee • Create a column family by name ○ Employee-Info with attributes ○ Emp_Id Primary Key, Emp_Name, Designation, ○ Date_of_Joining, Salary, Dept_Name • Insert the values into the table in batch • Update Employee name and Department of EmpId 121 • Sort the details of Employee records based on salary • Alter the schema of the table Employee_Info to add a column Projects which stores a set of Projects done by the corresponding Employee. • Update the altered table to add project names. • Create a TTL of 15 seconds to display the values of Employees.	4
3	16.02.26	Perform the following DB operations using Cassandra. • Create a keyspace by name Library • Create a column family by name Library-Info with attributes ○ Stud_Id Primary Key, ○ Counter_value of type Counter, ○ Stud_Name, Book-Name, Book-Id, ○ Date_of_issue • Insert the values into the table in batch • Display the details of the table created and increase the value of the counter • Write a query to show that a student with id 112 has taken a book “BDA” 2 times. • Export the created column to a csv file • g) Import a given csv dataset from local file system into Cassandra column family	6
4	23.02.26	Execution of HDFS Commands for interaction with Hadoop Environment. (Minimum 10 commands to be executed)	9
5	09.03.26	Implement Wordcount program on Hadoop framework	11
6	16.03.26	From the following link extract the weather data https://github.com/tomwhite/hadoop-book/tree/master/input/ncdc/all • Create a MapReduce program to find average temperature for each year from NCDC data set. • b) find the mean max temperature for every month	14

7	23.03.26	For a given Text file, Create a Map Reduce program to sort the content in an alphabetic order listing only top 10 maximum occurrences of words.	20
8	20.04.26	Write a Scala program to print numbers from 1 to 100 using for loop.	24
9	21.04.26	Using RDD and FlatMap count how many times each word appears in a file and write out a list of words whose count is strictly greater than 4 using Spark.	25
10	21.04.26	Write a simple streaming program in Spark to receive text data streams on a particular port, perform basic text cleaning (like white space removal, stop words removal, lemmatization, etc.), and print the cleaned text on the screen. (Open Ended Question).	27

Course Outcomes (COs):

CO1	Apply the concept of NoSQL, Hadoop or Spark for a given task
CO2	Analyse big data analytics mechanisms that can be applied to obtain solution for a given problem.
CO3	Design and implement solutions using data analytics mechanisms for a given problem.

Github link: <https://github.com/thanmaykh/bda->

LABORATORY PROGRAM – 1

MongoDB- CRUD Operations Demonstration (Practice and Self Study)

Command and Output

```
Atlas atlas-wanmtx-shard-0 [primary] Students> db.students.find({}, { "Name": 1, "grade":
{ $ifNull: ["$grade", "Not available"] }, "_id": 0 })
[
  { Name: 'John', grade: 'A' },
  { Name: 'Alice', grade: 'B' },
  { Name: 'Bob', grade: 'C' },
  { Name: 'Eve', grade: 'A' },
  { Name: 'Charlie', grade: 'Not available' }
]
```

```
Atlas atlas-wanmtx-shard-0 [primary] Students> db.students.find({ "hobby": { $nin: ["Chess", "Skating"] } })
[
  {
    _id: ObjectId("661ce9dc76a00ff8cc51dae1"),
    Rollno: 10,
    Name: 'John',
    Age: 20,
    ContactNo: '1234567890',
    'Email-Id': 'john.doe@example.com',
    grade: 'A',
    hobby: 'Reading'
  },
  {
    _id: ObjectId("661ce9dc76a00ff8cc51dae2"),
    Rollno: 11,
    Name: 'Alice',
    Age: 21,
    ContactNo: '9876543210',
    'Email-Id': 'alice@example.com',
    grade: 'B',
    hobby: 'Painting'
  },
  {
    _id: ObjectId("661ce9dc76a00ff8cc51dae3"),
    Rollno: 12,
    Name: 'Bob',
    Age: 22,
    ContactNo: '2345678901',
    'Email-Id': 'bob@example.com',
    grade: 'C',
    hobby: 'Cooking'
  },
]
```

```
Atlas atlas-wanmtx-shard-0 [primary] Students> db.students.updateOne(
...   { "Rollno": 11 },
...   { $set: { "Name": "Alice" } }
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
Atlas atlas-wanmtx-shard-0 [primary] Students> db.students.updateOne(
...   { "Rollno": 10 },
...   { $set: { "Email-Id": "john.doe@example.com" } }
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```

Atlas atlas-wanmtx-shard-0 [primary] Student> use Students
switched to db Students
Atlas atlas-wanmtx-shard-0 [primary] Students> show collections

Atlas atlas-wanmtx-shard-0 [primary] Students> db.students.insertMany([
...   { "Rollno": 10, "Name": "John", "Age": 20, "ContactNo": "1234567890", "Email-Id":
"john@example.com", "grade": "A", "hobby": "Reading" },
...   { "Rollno": 11, "Name": "Alice", "Age": 21, "ContactNo": "9876543210", "Email-Id":
"alice@example.com", "grade":
"B", "hobby": "Painting" },
...   { "Rollno": 12, "Name": "Bob", "Age": 22, "ContactNo": "2345678901", "Email-Id": "
bob@example.com", "grade": "C", "hobby": "Cooking" },
...   { "Rollno": 13, "Name": "Eve", "Age": 23, "ContactNo": "3456789012", "Email-Id": "
eve@example.com", "grade": "A"
},
...   { "Rollno": 14, "Name": "Charlie", "Age": 24, "ContactNo": "4567890123", "Email-Id
": "charlie@example.com", "hobby": "Gardening" }
... ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId("661ce9dc76a00ff8cc51dae1"),
    '1': ObjectId("661ce9dc76a00ff8cc51dae2"),
    '2': ObjectId("661ce9dc76a00ff8cc51dae3"),
    '3': ObjectId("661ce9dc76a00ff8cc51dae4"),
    '4': ObjectId("661ce9dc76a00ff8cc51dae5")
  }
}

```

LABORATORY PROGRAM – 2

Perform the following DB operations using Cassandra

Questions:

- Create a keyspace by name Employee
- Create a column family by name
 - Employee-Info with attributes
 - Emp_Id Primary Key, Emp_Name,
 - Designation, Date_of_Joining,
 - Salary, Dept_Name
- Insert the values into the table in batch
- Update Employee name and Department of Emp-Id 121
- Sort the details of Employee records based on salary
- Alter the schema of the table Employee_Info to add a column Projects which stores a set of Projects done by the corresponding Employee.
- Update the altered table to add project names.
- Create a TTL of 15 seconds to display the values of Employees.

COMMAND WITH OUTPUT

```
connected@braccae-MP-Elite-Tower-880-GX-Desktop-PC:~$ cqlsh
Connected to 'test_cluster' at 127.0.0.1:9042
[cqlsh 0.1.0 | Cassandra 4.1.4 | CQL spec 3.4.0 | Native protocol v5]
Use HELP for help.
cqlsh> CREATE KEYSPACE Students WITH REPLICATION={
... 'class':'SimpleStrategy','replication_factor':1};
cqlsh> DESCRIBE KEYSPACES

students      system_auth      system_schema      system_views
system        system_distributed system_traces       system_virtual_schema

cqlsh> SELECT * FROM system.schema_keyspaces;
UnavailableRequest: Error from server: code=2100 [Invalid query] message="table schema_keyspaces does not exist"
cqlsh> use Students;
cqlsh:students> create table Students_Info(Roll_No int Primary Key,StudName text,DateofJoining timestamp,last_exam_Percent double);
cqlsh:students> describe tables;

students_info

cqlsh:students> describe table students;
Table 'students' not found in keyspace 'students'
cqlsh:students> describe table students_info;

CREATE TABLE students.students_info (
  roll_no int PRIMARY KEY,
  dateofjoining timestamp,
  last_exam_percent double,
  studname text
) WITH additional_write_policy = '99p'
AND bloom_filter_fp_chance = 0.01
AND caching = {'keys': 'ALL', 'rows_per_partition': 'NONE'}
AND cdc = false
AND comment = ''
AND compaction = {'class': 'org.apache.cassandra.db.compaction.SizeTieredCompactionStrategy', 'max_threshold': '32', 'min_threshold': '4'}
AND compression = {'chunk_length_in_kb': '16', 'class': 'org.apache.cassandra.io.compress.LZ4Compressor'}
AND memtable = 'default'
AND crc_check_chance = 1.0
AND default_time_to_live = 0
AND extensions = {}
AND gc_grace_seconds = 864000
AND max_index_interval = 2048
AND memtable_flush_period_in_ms = 0
AND min_index_interval = 128
AND read_repair = 'BLOCKING'
AND speculative_retry = '99p';
```



```

sqlsh>students> Begin batch insert into Students_Info(Roll_no, StudName,DateOfBirth, Last_exam_Percent) values(1,'Sudhansu','2023-10-09', 88) insert into Students_Info(Roll_no, StudName,DateOfBirth, Last_exam_Percent) values(2,'Rita','2023-10-18', 87) insert into Students_Info(Roll_no, StudName,DateOfBirth, Last_exam_Percent) values(3,'Rachana','2023-10-18', 87.5) insert into Students_Info(Roll_no, StudName,DateOfBirth, Last_exam_Percent) values(4,'Charu','2023-10-06', 86.9) apply batch;
sqlsh>students> select * from students_info;

Roll_no | DateOfBirth | Last_exam_Percent | StudName
-----|-----|-----|-----
1 | 2023-10-09 10:30:00.000000+0000 | 88 | Sudhansu
2 | 2023-10-18 10:30:00.000000+0000 | 87 | Rita
3 | 2023-10-18 10:30:00.000000+0000 | 87.5 | Rachana
4 | 2023-10-06 10:30:00.000000+0000 | 86.9 | Charu

(4 rows)
sqlsh>students> select * from students_info where roll_no in (1,2,3);

Roll_no | DateOfBirth | Last_exam_Percent | StudName
-----|-----|-----|-----
1 | 2023-10-09 10:30:00.000000+0000 | 88 | Sudhansu
2 | 2023-10-18 10:30:00.000000+0000 | 87 | Rita
3 | 2023-10-18 10:30:00.000000+0000 | 87.5 | Rachana

(3 rows)
sqlsh>students> select * from students_info where StudName='Charu';
Warning: [Warning] Using Char keyword (modified SQL92) might degrade query performance and may have undesirable performance. If you want to preserve this query & reduce the performance impact, use SQL92 syntax.
sqlsh>students> create index on Students_Info(StudName);
sqlsh>students> select * from students_info where StudName='Charu';

Roll_no | DateOfBirth | Last_exam_Percent | StudName
-----|-----|-----|-----
4 | 2023-10-06 10:30:00.000000+0000 | 86.9 | Charu

(1 rows)
sqlsh>students> select Roll_no,StudName from students_info LIMIT 2;

```

```

(4 rows)
sqlsh>students> select * from students_info where roll_no in (1,2,3);

Roll_no | DateOfBirth | Last_exam_Percent | StudName
-----|-----|-----|-----
1 | 2023-10-09 10:30:00.000000+0000 | 88 | Sudhansu
2 | 2023-10-18 10:30:00.000000+0000 | 87 | Rita
3 | 2023-10-18 10:30:00.000000+0000 | 87.5 | Rachana

(3 rows)
sqlsh>students> select * from students_info where StudName='Charu';
Warning: [Warning] Using Char keyword (modified SQL92) might degrade query performance and may have undesirable performance. If you want to preserve this query & reduce the performance impact, use SQL92 syntax.
sqlsh>students> create index on Students_Info(StudName);
sqlsh>students> select * from students_info where StudName='Charu';

Roll_no | DateOfBirth | Last_exam_Percent | StudName
-----|-----|-----|-----
4 | 2023-10-06 10:30:00.000000+0000 | 86.9 | Charu

(1 rows)
sqlsh>students> select Roll_no,StudName from students_info LIMIT 2;

Roll_no | StudName
-----|-----
1 | Sudhansu
2 | Rita

(2 rows)
sqlsh>students> SELECT Roll_no as "ID" from Students_Info;

ID
--
1
2
3
4

```

LABORATORY PROGRAM – 3

Perform the following DB operations using Cassandra

Questions:

- Create a keyspace by name Library
- Create a column family by name Library-Info with attributes
 - Stud_Id Primary Key,
 - Counter_value of type Counter,
 - Stud_Name, Book-Name, Book-Id,
 - Date_of_issue
- Insert the values into the table in batch
- Display the details of the table created and increase the value of the counter
- Write a query to show that a student with id 112 has taken a book “BDA” 2 times.
- Export the created column to a csv file
- Import a given csv dataset from local file system into Cassandra column family

COMMAND WITH OUTPUT

```
cqlsh:employee> CREATE KEYSPACE IF NOT EXISTS Library
... WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};
cqlsh:employee> USE Library;
cqlsh:library> CREATE TABLE IF NOT EXISTS Library_Info (
...     Stud_Id INT PRIMARY KEY,
...     Stud_Name TEXT,
...     Book_Name TEXT,
...     Book_Id TEXT,
...     Date_of_issue DATE
... );
cqlsh:library> CREATE TABLE IF NOT EXISTS Book_Counter (
...     Stud_Id INT,
...     Book_Name TEXT,
...     Counter_value COUNTER,
...     PRIMARY KEY ((Stud_Id), Book_Name)
... );
cqlsh:library> BEGIN BATCH
... INSERT INTO Library_Info (Stud_Id, Stud_Name, Book_Name, Book_Id, Date_of_issue)
... VALUES (112, 'Anjali Rao', 'BDA', 'B101', '2024-10-01');
... INSERT INTO Library_Info (Stud_Id, Stud_Name, Book_Name, Book_Id, Date_of_issue)
... VALUES (113, 'Marthik N', 'AI', 'B102', '2024-11-11');
... APPLY BATCH;
cqlsh:library> UPDATE Book_Counter SET Counter_value = Counter_value + 1 WHERE Stud_Id = 112 AND Book_Name = 'BDA';
cqlsh:library> UPDATE Book_Counter SET Counter_value = Counter_value + 1 WHERE Stud_Id = 112 AND Book_Name = 'BDA';
cqlsh:library> SELECT * FROM Book_Counter WHERE Stud_Id = 112 AND Book_Name = 'BDA';

stud_id | book_name | counter_value
-----+-----+-----
112    | BDA      | 4
(1 rows)
```

```

cqlsh:students> DESCRIBE TABLE Students_Info;

CREATE TABLE students.students_info (
  roll_no int PRIMARY KEY,
  dateofjoining timestamp,
  last_exam_percent double,
  studname text
) WITH additional_write_policy = '99p'
AND bloom_filter_fp_chance = 0.01
AND caching = {'keys': 'ALL', 'rows_per_partition': 'NONE'}
AND cdc = false
AND comment = ''
AND compaction = {'class': 'org.apache.cassandra.db.compaction.SizeTieredCompactionStrategy', 'max_threshold': '32', 'min_threshold': '4'}
AND compression = {'chunk_length_in_kb': '16', 'class': 'org.apache.cassandra.io.compress.LZ4Compressor'}
AND mentable = 'default'
AND crc_check_chance = 1.0
AND default_time_to_live = 0
AND extensions = {}
AND gc_grace_seconds = 864000
AND max_index_interval = 2548
AND memtable_flush_period_in_ms = 0
AND min_index_interval = 128
AND read_repair = 'BLOCKING'
AND speculative_retry = '99p';
cqlsh:students> BEGIN BATCH
... INSERT INTO Students_Info (Roll_No, StudName, DateOfJoining, Last_Exam_Percent)
... VALUES (1, 'Asha', '2012-03-12', 79.9);
... INSERT INTO Students_Info (Roll_No, StudName, DateOfJoining, Last_Exam_Percent)
... VALUES (2, 'Kiran', '2012-03-12', 89.9);
... INSERT INTO Students_Info (Roll_No, StudName, DateOfJoining, Last_Exam_Percent)
... VALUES (3, 'Shanthi', '2012-03-12', 90.9);
... INSERT INTO Students_Info (Roll_No, StudName, DateOfJoining, Last_Exam_Percent)
... VALUES (4, 'Smith', '2012-03-12', 87.9);
... INSERT INTO Students_Info (Roll_No, StudName, DateOfJoining, Last_Exam_Percent)
... VALUES (5, 'Rohan', '2012-03-12', 50.9);
... APPLY BATCH;
cqlsh:students> SELECT * FROM Students_Info;

roll_no | dateofjoining | last_exam_percent | studname
-----|-----|-----|-----
1 | 2012-03-12 10:30:00.000000+0000 | 79.9 | Asha
2 | 2012-03-12 10:30:00.000000+0000 | 89.9 | Kiran
4 | 2012-03-12 10:30:00.000000+0000 | 87.9 | Smith
3 | 2012-03-12 10:30:00.000000+0000 | 90.9 | Shanthi
5 | 2012-03-12 10:30:00.000000+0000 | 50.9 | Rohan

(5 rows)

```

```

cqlsh> CREATE KEYSPACE Students WITH REPLICATION =
... {'class': 'SimpleStrategy', 'replication_factor': '1'};
cqlsh>
cqlsh> USE Students;
cqlsh:students> DESCRIBE KEYSPACES;

companies | library | products | system | system_traces
company | pro | productss | system_auth | system_views
employe | prod | productsss | system_distributed | system_virtual_schema
employee | productname | students | system_scheme

cqlsh:students> CREATE TABLE Students_Info (
... Roll_No int PRIMARY KEY,
... StudName text,
... DateOfJoining timestamp,
... Last_Exam_Percent double
... );
cqlsh:students> SELECT * FROM system.schema_keyspaces;
InvalidRequest: Error from server: code=2200 [invalid query] message="table schema_keyspaces does not exist"
cqlsh:students>
cqlsh:students> SELECT * FROM system_schema.keyspaces;

keyspace_name | durable_writes | replication
-----|-----|-----
companies | True | {'class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '1'}
system_auth | True | {'class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '1'}
system_schema | True | {'class': 'org.apache.cassandra.locator.LocalStrategy'}
library | True | {'class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '1'}
products | True | {'class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '1'}
system_distributed | True | {'class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '1'}
system | True | {'class': 'org.apache.cassandra.locator.LocalStrategy'}
productsss | True | {'class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '1'}
prod | True | {'class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '1'}
pro | True | {'class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '1'}
system_traces | True | {'class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '1'}
students | True | {'class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '1'}
company | True | {'class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '1'}
employee | True | {'class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '1'}
productname | True | {'class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '1'}
employe | True | {'class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '1'}
products | True | {'class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '1'}

(17 rows)
cqlsh:students> DESCRIBE TABLES;

students_info

```

```

cqlsh:students> SELECT * FROM Students_Info WHERE Roll_No IN (1,2,3);

roll_no | dateofjoining | last_exam_percent | studname
-----+-----+-----+-----
1 | 2012-03-11 18:30:00.000000+0000 | 79.9 | Asha
2 | 2012-03-11 18:30:00.000000+0000 | 89.9 | Kiran
3 | 2012-03-11 18:30:00.000000+0000 | 90.9 | Shanthi

(3 rows)
cqlsh:students> CREATE INDEX ON Students_Info (StudName);
cqlsh:students> SELECT * FROM Students_Info WHERE StudName = 'Asha';

roll_no | dateofjoining | last_exam_percent | studname
-----+-----+-----+-----
1 | 2012-03-11 18:30:00.000000+0000 | 79.9 | Asha

(1 rows)
cqlsh:students> SELECT Roll_No, StudName FROM Students_Info LIMIT 2;

roll_no | studname
-----+-----
5 | Rohan
1 | Asha

(2 rows)
cqlsh:students> SELECT Roll_No AS USN FROM Students_Info;

USN
----
5
1
2
4
3

(5 rows)
cqlsh:students> UPDATE Students_Info
... SET StudName = 'David Sheen'
... WHERE Roll_No = 2;
cqlsh:students> UPDATE Students_Info SET Roll_No = 0 WHERE Roll_No = 3; -- X ERROR!
InvalidRequest: Error from server: code=2200 [Invalid query] message="PRIMARY KEY part roll_no found in SET part"

```

```

cqlsh:students> DELETE Last_Exam_Percent FROM Students_Info WHERE Roll_No = 2;
cqlsh:students> DELETE FROM Students_Info WHERE Roll_No = 2;
cqlsh:students> ALTER TABLE Students_Info ADD hobbies SET<text>;
cqlsh:students> ALTER TABLE Students_Info ADD languages LIST<text>;
cqlsh:students> UPDATE Students_Info
... SET hobbies = hobbies + {'Chess', 'Table Tennis'}
... WHERE Roll_No = 1;
cqlsh:students> CREATE TABLE library_book (
... counter_value counter,
... book_name text,
... stud_name text,
... PRIMARY KEY(book_name, stud_name)
... );
cqlsh:students> UPDATE library_book
... SET counter_value = counter_value + 1
... WHERE book_name = 'Big Data Analytics' AND stud_name = 'Jeet';
cqlsh:students> CREATE TABLE userlogin (
... userid int PRIMARY KEY,
... password text
... );
cqlsh:students> INSERT INTO userlogin (userid, password)
... VALUES (1, 'info') USING TTL 30;
cqlsh:students> SELECT TTL(password) FROM userlogin WHERE userid = 1;

TTL(password)
-----
30

(1 rows)
cqlsh:students> COPY Students_Info TO '/home/bmsccse/Desktop/Student_Info.csv';
Using 16 child processes

Starting copy of students.students_info with columns [roll_no, dateofjoining, hobbies, languages, last_exam_percent, studname].
Processed: 4 rows; Rate: 38 rows/s; Avg. rate: 38 rows/s
4 rows exported to 1 files in 0.124 seconds.
cqlsh:students> COPY Students_Info FROM '/home/bmsccse/Desktop/Student_Info.csv';
Using 16 child processes

Starting copy of students.students_info with columns [roll_no, dateofjoining, hobbies, languages, last_exam_percent, studname].
Processed: 4 rows; Rate: 7 rows/s; Avg. rate: 11 rows/s
4 rows imported from 1 files in 0.377 seconds (0 skipped).
cqlsh:students> COPY person (id, fname, lname) FROM STDIN;
id,fname,lname
-----
3,2012-03-11 18:30:00.000+0000,,,50.9,Rohan
1,2012-03-11 18:30:00.000+0000,{'Chess', 'Table Tennis'},,79.9,Asha
4,2012-03-11 18:30:00.000+0000,,,67.9,Smith
3,2012-03-11 18:30:00.000+0000,,,90.9,Shanthi
cqlsh:students>

```


LABORATORY PROGRAM – 4

Execution of HDFS Commands for interaction with Hadoop Environment. (Minimum 10 commands to be executed)

COMMAND WITH OUTPUT

```
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~$ cd ./Desktop/
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ start-all.sh
WARNING: Attempting to start all Apache Hadoop daemons as hadoop in 10 seconds.
WARNING: This is not a recommended production deployment configuration.
WARNING: Use CTRL-C to abort.
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [bmscscse-HP-Elite-Tower-800-G9-Desktop-PC]
Starting resourcemanager
Starting nodemanagers
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -mkdir /Lab05
```

```
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -ls /Hadoop
ls: '/Hadoop': No such file or directory
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -ls /Lab05
```

```
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ touch test.txt
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ nano test.txt
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -put ./text.txt /Lab05/text.txt
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -ls /Lab05
Found 1 items
-rw-r--r-- 1 hadoop supergroup      19 2024-05-13 14:33 /Lab05/text.txt
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -cat /Lab05/text.txt
Hello
How are you?
```

```
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -ls /Lab05
Found 2 items
-rw-r--r-- 1 hadoop supergroup      15 2024-05-13 14:40 /Lab05/test.txt
-rw-r--r-- 1 hadoop supergroup      19 2024-05-13 14:33 /Lab05/text.txt
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -getmerge /Lab05 /text.txt /Lab05 /test.txt ../
Downloads/Merged.txt
getmerge: '/text.txt': No such file or directory
getmerge: '/test.txt': No such file or directory
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -getmerge /Lab05/text.txt /Lab05/test.txt ../Do
wnloads/Merged.txt
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -getfacl /Lab05
# file: /Lab05
# owner: hadoop
# group: supergroup
user::rwx
group::r-x
other::r-x
```

```
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -put /home/hadoop/Desktop/Welcome.txt /abc/WC.txt
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -copyFromLocal /home/hadoop/Desktop/Welcome.txt /abc/WC.txt
copyFromLocal: '/abc/WC.txt': File exists
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -get /abc/WC.txt /home/hadoop/Downloads/MWC.txt
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -getmerge /abc/ /home/hadoop/Desktop/Merge.txt
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -getfacl /abc/
# file: /abc
# owner: hadoop
# group: supergroup
user::rwx
group::r-x
other::r-x
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -cat /abc/WC.txt
hello world
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -mv /abc /FFF
hadoop fs -ls /FFF
Found 3 items
-rw-r--r-- 1 hadoop supergroup      12 2025-04-15 14:53 /FFF/WC.txt
-rw-r--r-- 1 hadoop supergroup      12 2024-05-14 14:35 /FFF/file.txt
-rw-r--r-- 1 hadoop supergroup      12 2024-05-14 14:38 /FFF/file_cp_local.txt
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -cp /CSE/ /LLL
```

```
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -cat /Lab05/text.txt
Hello
How are you?
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -mv /Lab05 /test_Lab05
```

```
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -ls /test_Lab05
Found 2 items
-rw-r--r-- 1 hadoop supergroup 15 2024-05-13 14:40 /test_Lab05/test.txt
-rw-r--r-- 1 hadoop supergroup 19 2024-05-13 14:33 /test_Lab05/text.txt
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -cp /test_Lab05/ /Lab05
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -ls /Lab05
Found 2 items
-rw-r--r-- 1 hadoop supergroup 15 2024-05-13 14:51 /Lab05/test.txt
-rw-r--r-- 1 hadoop supergroup 19 2024-05-13 14:51 /Lab05/text.txt
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -ls /test_Lab05
Found 2 items
-rw-r--r-- 1 hadoop supergroup 15 2024-05-13 14:40 /test_Lab05/test.txt
-rw-r--r-- 1 hadoop supergroup 19 2024-05-13 14:33 /test_Lab05/text.txt
```

LABORATORY PROGRAM – 5

Implement Wordcount program on Hadoop framework

CODE

Driver Code

```
import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.FileInputFormat;
import org.apache.hadoop.mapred.FileOutputFormat;
import org.apache.hadoop.mapred.JobClient;
import org.apache.hadoop.mapred.JobConf;
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;
public class WCDriver extends Configured implements Tool {

    public int run(String[] args) throws IOException {
        if (args.length < 2) {
            System.out.println("Please give valid inputs");
            return -1;
        }

        JobConf conf = new JobConf(WCDriver.class);
        conf.setJobName("WordCount");

        FileInputFormat.setInputPaths(conf, new Path(args[0]));
        FileOutputFormat.setOutputPath(conf, new Path(args[1]));

        conf.setMapperClass(WCMapper.class);
        conf.setReducerClass(WCReducer.class);

        conf.setMapOutputKeyClass(Text.class);
        conf.setMapOutputValueClass(IntWritable.class);

        conf.setOutputKeyClass(Text.class);
        conf.setOutputValueClass(IntWritable.class);

        JobClient.runJob(conf);
        return 0;
    }

    // Main Method
    public static void main(String[] args) throws Exception {
        int exitCode = ToolRunner.run(new WCDriver(), args);
        System.out.println("Job Exit Code: " + exitCode);
    }
}
```

Mapper Code

```
// Importing libraries import
java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import
org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.MapReduceBase;
import org.apache.hadoop.mapred.Mapper;
import org.apache.hadoop.mapred.OutputCollector;
import org.apache.hadoop.mapred.Reporter;
```

```
public class WCMapper extends MapReduceBase implements Mapper<LongWritable, Text, Text, IntWritable> {

    // Map function
    public void map(LongWritable key, Text value, OutputCollector<Text, IntWritable> output, Reporter reporter)
        throws IOException {
        String line = value.toString();

        // Splitting the line on whitespace
        for (String word : line.split("\\s+")) {
            if (word.length() > 0) {
                output.collect(new Text(word), new IntWritable(1));
            }
        }
    }
}
```

Reducer Code

```
// Importing libraries import
java.io.IOException; import
java.util.Iterator;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.MapReduceBase;
import org.apache.hadoop.mapred.OutputCollector;
import org.apache.hadoop.mapred.Reducer;
import org.apache.hadoop.mapred.Reporter;

public class WCReducer extends MapReduceBase implements Reducer<Text, IntWritable, Text, IntWritable> {

    // Reduce function
    public void reduce(Text key, Iterator<IntWritable> values,
        OutputCollector<Text, IntWritable> output,
        Reporter reporter) throws IOException {
        int count = 0;

        // Counting the frequency of each word
        while (values.hasNext()) {
            count += values.next().get();
        }

        output.collect(key, new IntWritable(count));
    }
}
```

COMMAND WITH OUTPUT

[illegible]

LABORATORY PROGRAM – 6

Implement Weather program on Hadoop framework

Questions:

From the following link extract the weather data

<https://github.com/tomwhite/hadoopbook/tree/master/input/ncdc/all>

- a) Create a MapReduce program to find average temperature for each year from NCDC data set.
- b) find the mean max temperature for every month.

A) CODE

```
import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import
org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class AverageDriver {
    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.err.println("Please enter both input and output
parameters."); System.exit(-1);
        }

        // Creating a configuration and job instance
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Average Calculation");

        job.setJarByClass(AverageDriver.class);

        // Input and output paths
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        // Setting mapper and reducer classes
        job.setMapperClass(AverageMapper.class);
        job.setReducerClass(AverageReducer.class);

        // Output key and value types
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        // Submitting the job and waiting for it to complete
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

```
import org.apache.hadoop.conf.Configuration;
```

Mapper Code

```
package temp;

import java.io.IOException;

import org.apache.hadoop.io.IntWritable;
import
org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class AverageMapper extends Mapper<LongWritable, Text, Text, IntWritable> {

    public static final int MISSING = 9999;

    @Override
    public void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {

        String line = value.toString();

        // Extract year from fixed position
        String year = line.substring(15, 19);
        int temperature;

        // Determine if there's a '+' sign
        if (line.charAt(87) == '+') {
            temperature = Integer.parseInt(line.substring(88, 92));
        } else {
            temperature = Integer.parseInt(line.substring(87, 92));
        }

        // Quality check character
        String quality = line.substring(92, 93);

        // Only emit if data is valid
        if (temperature != MISSING && quality.matches("[01459]")) {
            context.write(new Text(year), new IntWritable(temperature));
        }
    }
}
```

Reducer Code

```
package temp;

import java.io.IOException;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class AverageReducer extends Reducer<Text, IntWritable, Text, IntWritable> {

    @Override
    public void reduce(Text key, Iterable<IntWritable> values,
        Context context) throws IOException, InterruptedException {

        int sumTemp = 0;
        int count = 0;

        for (IntWritable value : values) {
            sumTemp += value.get();
            count++;
        }

        if (count > 0) {
```

```

int average = sumTemp / count; context.write(key, new IntWritable(average));
}
}
}

```

A) COMMAND WITH OUTPUT

```

C:\hadoop-3.3.0\sbin>hadoop jar C:\avgtemp.jar temp.AverageDriver /input_dir/temp.txt /avgtemp_outputdir
2021-05-15 14:52:50,035 INFO client.DefaultHadoopWalloverProxy/Provider: Connecting to ResourceManager at /0.0.0.0:8032
2021-05-15 14:52:51,005 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool Interface and execute your application with ToolRunner to remedy this.
2021-05-15 14:52:51,111 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/Anusree/.staging/job_1621060230696_0005
2021-05-15 14:52:51,735 INFO InputFileInputFormat: Total input files to process : 1
2021-05-15 14:52:52,751 INFO mapreduce.JobSubmitter: number of splits:1
2021-05-15 14:52:53,073 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1621060230696_0005
2021-05-15 14:52:53,073 INFO mapreduce.JobSubmitter: Executing with tokens: []
2021-05-15 14:52:53,237 INFO conf.Configuration: resource-types.xml not found
2021-05-15 14:52:53,238 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2021-05-15 14:52:53,312 INFO impl.YarnClientImpl: Submitted application application_1621060230696_0005
2021-05-15 14:52:53,352 INFO mapreduce.Job: The url to track the job: http://LAPTOP-J6329ESD:8080/proxy/application_1621060230696_0005/
2021-05-15 14:52:53,353 INFO mapreduce.Job: Running job: job_1621060230696_0005
2021-05-15 14:53:06,040 INFO mapreduce.Job: Job job_1621060230696_0005 running in uber mode : false
2021-05-15 14:53:06,043 INFO mapreduce.Job: map 0% reduce 0%
2021-05-15 14:53:12,758 INFO mapreduce.Job: map 100% reduce 0%
2021-05-15 14:53:19,860 INFO mapreduce.Job: map 100% reduce 100%
2021-05-15 14:53:25,967 INFO mapreduce.Job: Job job_1621060230696_0005 completed successfully
2021-05-15 14:53:26,096 INFO mapreduce.Job: Counters: 54
  File System Counters
    FILE: Number of bytes read=72210
    FILE: Number of bytes written=674341
    FILE: Number of read operations=0
    FILE: Number of large read operations=0
    FILE: Number of write operations=0
    HDFS: Number of bytes read=994060
    HDFS: Number of bytes written=8
    HDFS: Number of read operations=8
    HDFS: Number of large read operations=0
    HDFS: Number of write operations=2
    HDFS: Number of bytes read erasure-coded=0
  Job Counters
    Launched map tasks=1
    Launched reduce tasks=1
    Data-local map tasks=1
    Total time spent by all maps in occupied slots (ms)=3782

```

```

C:\hadoop-3.3.0\sbin>hdfs dfs -ls /avgtemp_outputdir
Found 2 items
-rw-r--r--   1 Anusree supergroup          0 2021-05-15 14:53 /avgtemp_outputdir/_SUCCESS
-rw-r--r--   1 Anusree supergroup          8 2021-05-15 14:53 /avgtemp_outputdir/part-r-000000

C:\hadoop-3.3.0\sbin>hdfs dfs -cat /avgtemp_outputdir/part-r-000000
1901    46

C:\hadoop-3.3.0\sbin>

```

B) Code

```
package meanmax;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import
org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class MeanMaxDriver {
    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.err.println("Please enter both input and output
parameters."); System.exit(-1);
        }

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Mean and Max Temperature");

        job.setJarByClass(MeanMaxDriver.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        job.setMapperClass(MeanMaxMapper.class);
        job.setReducerClass(MeanMaxReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Mapper Code

```
package meanmax;

import java.io.IOException;

import org.apache.hadoop.io.IntWritable;
import
org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Mapper;

public class MeanMaxMapper extends Mapper<LongWritable, Text, Text, IntWritable> {

    public static final int MISSING = 9999;

    @Override
    public void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {

        String line = value.toString();

        // Extract month from positions 19-20
```

```

String month = line.substring(19, 21); int temperature;

// Extract temperature considering optional '+'
if (line.charAt(87) == '+') {
    temperature = Integer.parseInt(line.substring(88, 92));
} else {
    temperature = Integer.parseInt(line.substring(87, 92));
}

// Quality check
String quality = line.substring(92, 93);

if (temperature != MISSING && quality.matches("[01459]")) {
    context.write(new Text(month), new
        IntWritable(temperature));
}
}
}

```

Reducer Code

```

package meanmax;

import java.io.IOException;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;
public class MeanMaxReducer extends Reducer<Text, IntWritable, Text, Text> {
    @Override
    public void reduce(Text key, Iterable<IntWritable> values,
        Context context) throws IOException, InterruptedException {

        int sumTemp = 0;
        int count = 0;
        int maxTemp = Integer.MIN_VALUE;

        for (IntWritable value : values) {
            int temp = value.get();
            sumTemp += temp;
            count++;

            if (temp > maxTemp) {
                maxTemp = temp;
            }
        }

        if (count > 0) {
            int avgTemp = sumTemp / count;
            String result = "mean=" + avgTemp + " max=" + maxTemp;
            context.write(key, new Text(result));
        }
    }
}

```

B) COMMAND WITH OUTPUT

```
C:\hadoop-3.3.0\sbin>hadoop jar C:\meanmax.jar meanmax.MeanMaxDriver /input_dir/temp.txt /meanmax_output
2021-05-21 20:28:05,250 INFO client.DefaultHadoopFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2021-05-21 20:28:06,662 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
2021-05-21 20:28:06,916 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/Anusreef.staging/job_1621608943095_0001
2021-05-21 20:28:08,426 INFO input.FileInputFormat: Total input files to process : 1
2021-05-21 20:28:09,107 INFO mapreduce.JobSubmitter: number of splits:1
2021-05-21 20:28:09,741 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1621608943095_0001
2021-05-21 20:28:09,741 INFO mapreduce.JobSubmitter: Executing with tokens: []
2021-05-21 20:28:10,029 INFO conf.Configuration: resource-types.xml not found
2021-05-21 20:28:10,030 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2021-05-21 20:28:10,676 INFO impl.YarnClientImpl: Submitted application application_1621608943095_0001
2021-05-21 20:28:11,005 INFO mapreduce.Job: The url to track the job: http://LAPTOP-36329ESD:8088/proxy/application_1621608943095_0001/
2021-05-21 20:28:11,006 INFO mapreduce.Job: Running job: job_1621608943095_0001
2021-05-21 20:28:29,385 INFO mapreduce.Job: Job job_1621608943095_0001 running in uber mode : false
2021-05-21 20:28:29,389 INFO mapreduce.Job:  map 0% reduce 0%
2021-05-21 20:28:40,664 INFO mapreduce.Job:  map 100% reduce 0%
2021-05-21 20:28:50,832 INFO mapreduce.Job:  map 100% reduce 100%
2021-05-21 20:28:58,965 INFO mapreduce.Job: Job job_1621608943095_0001 completed successfully
2021-05-21 20:28:59,178 INFO mapreduce.Job: Counters: 54
  File System Counters
    FILE: Number of bytes read=50082
    FILE: Number of bytes written=640091
    FILE: Number of read operations=0
    FILE: Number of large read operations=0
    FILE: Number of write operations=0
    HDFS: Number of bytes read=894860
    HDFS: Number of bytes written=74
    HDFS: Number of read operations=0
    HDFS: Number of large read operations=0
    HDFS: Number of write operations=2
    HDFS: Number of bytes read erasure-coded=0
  Job Counters
    Launched map tasks=1
    Launched reduce tasks=1
    Data-local map tasks=1
    Total time spent by all maps in occupied slots (ms)=8077
    Total time spent by all reduces in occupied slots (ms)=7511
    Total time spent by all map tasks (ms)=8077
    Total time spent by all reduce tasks (ms)=7511
    Total vcore-milliseconds taken by all map tasks=8077
    Total vcore-milliseconds taken by all reduce tasks=7511
    Total megabyte-milliseconds taken by all map tasks=8270848
    Total megabyte-milliseconds taken by all reduce tasks=7691264
```

```
C:\hadoop-3.3.0\sbin>hdfs dfs -cat /meanmax_output/*
```

```
01      4
02      0
03      7
04     44
05    100
06    168
07    219
08    198
09    141
10    100
11     19
12      3
```

```
C:\hadoop-3.3.0\sbin>
```


LABORATORY PROGRAM – 7

For a given Text file, Create a Map Reduce program to sort the content in an alphabetic order listing only top 10 maximum occurrences of words.

Code

Mapper Code (WordCountMapper.java)

```
package samples.topn;

import java.io.IOException;
import java.util.StringTokenizer;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class WordCountMapper
    extends Mapper<Object, Text, Text, IntWritable> {

    private final static IntWritable ONE = new IntWritable(1);
    private Text word = new Text();
    // characters to normalize into spaces
    private String tokens = "[_!$#<>\\^=\\[\\]\\|*\\/\\\\\\.,;.:\\-:()?!\"'"]";

    @Override
    protected void map(Object key, Text value, Context context)
        throws IOException, InterruptedException {

        // clean & tokenize
        String clean = value.toString()
            .toLowerCase()
            .replaceAll(tokens, " ");
        StringTokenizer itr = new StringTokenizer(clean);
        while (itr.hasMoreTokens()) {
            word.set(itr.nextToken().trim());
            context.write(word, ONE);
        }
    }
}
```

Mapper Code (TopNMapper.java)

```
package samples.topn;

import java.io.IOException;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class TopNMapper
    extends Mapper<Object, Text, IntWritable, Text> {

    private IntWritable count = new IntWritable();
    private Text word = new Text();

    @Override
    protected void map(Object key, Text value, Context context)
        throws IOException, InterruptedException {
```

```
// input line: word \t count  
String[] parts = value.toString().split("\\t");
```

```

        if (parts.length == 2) {
            word.set(parts[0]);
            count.set(Integer.parseInt(parts[1]));
            // emit count → word, so Hadoop sorts by count
            context.write(count, word);
        }
    }
}

```

Reducer Code (WordCountReducer.java)

```

package samples.topn; import
java.io.IOException import
org.apache.hadoop.io.IntWritable; import
org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class WordCountReducer
    extends Reducer<Text, IntWritable, Text, IntWritable> {

    @Override
    protected void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException {

        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }
        context.write(key, new IntWritable(sum))
    }
}

```

Reducer Code (TopNReducer.java)

```

package samples.topn;

import java.io.IOException;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class TopNReducer
    extends Reducer<IntWritable, Text, Text, IntWritable> {

    // TreeMap with descending order of keys (counts)
    private TreeMap<Integer, List<String>> countMap =
        new TreeMap<>(Collections.reverseOrder());

    @Override
    protected void reduce(IntWritable key, Iterable<Text> values, Context context)
        throws IOException, InterruptedException {

        int cnt = key.get();
        List<String> words = countMap.getOrDefault(cnt, new ArrayList<>());
        for (Text w : values) {
            words.add(w.toString());
        }
        countMap.put(cnt, words);
    }

    @Override
    protected void cleanup(Context context)
        throws IOException, InterruptedException {

        // collect top 10 word→count pairs
        List<WordCount> topList = new
        ArrayList<>(); int seen = 0;
    }
}

```

```

for (Map.Entry<Integer, List<String>> entry : countMap.entrySet()) {
    int cnt = entry.getKey();
    for (String w : entry.getValue()) {
        topList.add(new WordCount(w, cnt));
        seen++;
        if (seen == 10) break;
    }
}

// sort these 10 entries alphabetically by word
Collections.sort(topList, (a, b) -> a.word.compareTo(b.word));

// emit final top 10 in alphabetical order
for (WordCount wc : topList) {
    context.write(new Text(wc.word), new IntWritable(wc.count));
}
}

// helper class
private static class WordCount {
    String word;
    int count;
    WordCount(String w, int c) { word = w; count = c; }
}
}

```

COMMAND WITH OUTPUT

```

C:\hadoop-3.3.0\sbin>jps
11072 DataNode
20528 Jps
5620 ResourceManager
15532 NodeManager
6140 NameNode

C:\hadoop-3.3.0\sbin>hdfs dfs -mkdir /input_dir

C:\hadoop-3.3.0\sbin>hdfs dfs -ls /
Found 1 items
drwxr-xr-x  - Anusree supergroup      0 2021-05-08 19:46 /input_dir

C:\hadoop-3.3.0\sbin>hdfs dfs -copyFromLocal C:\input.txt /input_dir

C:\hadoop-3.3.0\sbin>hdfs dfs -ls /input_dir
Found 1 items
-rw-r--r--  1 Anusree supergroup     36 2021-05-08 19:48 /input_dir/input.txt

C:\hadoop-3.3.0\sbin>hdfs dfs -cat /input_dir/input.txt
hello
world
hello
hadoop
bye

```

```

C:\hadoop-3.3.0\sbin>hadoop jar C:\sort.jar samples.topn.TopN /input_dir/input.txt /output_dir
2021-05-08 19:54:54,582 INFO client.DefaultHadoopFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2021-05-08 19:54:55,291 INFO mapreduce.jobhdfsloader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/Anonymous/staging/job_1620483374279_0001
2021-05-08 19:54:55,821 INFO input.FileInputFormat: Total input files to process : 1
2021-05-08 19:54:56,261 INFO mapreduce.JobSubmitter: number of splits:1
2021-05-08 19:54:56,551 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1620483374279_0001
2021-05-08 19:54:56,552 INFO mapreduce.JobSubmitter: Executing with tokens: []
2021-05-08 19:54:56,841 INFO conf.Configuration: resource-types.xml not found
2021-05-08 19:54:56,843 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2021-05-08 19:54:57,087 INFO impl.YarnClientImpl: Submitted application application_1620483374279_0001
2021-05-08 19:54:57,507 INFO mapreduce.Job: The url to track the job: http://LAPTOP-3G129550-8888/proxy/application_1620483374279_0001/
2021-05-08 19:54:57,508 INFO mapreduce.Job: Running job: job_1620483374279_0001
2021-05-08 19:55:11,792 INFO mapreduce.Job: Job job_1620483374279_0001 running in uber mode : false
2021-05-08 19:55:13,794 INFO mapreduce.Job:  map 0% reduce 0%
2021-05-08 19:55:20,029 INFO mapreduce.Job:  map 100% reduce 0%
2021-05-08 19:55:27,116 INFO mapreduce.Job:  map 100% reduce 100%
2021-05-08 19:55:33,199 INFO mapreduce.Job: Job job_1620483374279_0001 completed successfully
2021-05-08 19:55:33,134 INFO mapreduce.Job: Counters: 54
    File System Counters
      FILE: Number of bytes read=65
      FILE: Number of bytes written=530397
      FILE: Number of read operations=0
      FILE: Number of large read operations=0
      FILE: Number of write operations=0
      HDFS: Number of bytes read=162
      HDFS: Number of bytes written=11
      HDFS: Number of read operations=4
      HDFS: Number of large read operations=0
      HDFS: Number of write operations=2
      HDFS: Number of bytes read erasure-coded=0

```

```

C:\hadoop-3.3.0\sbin>hdfs dfs -cat /output_dir/*
hello 2
hadoop 1
world 1
bye 1

C:\hadoop-3.3.0\sbin>

```

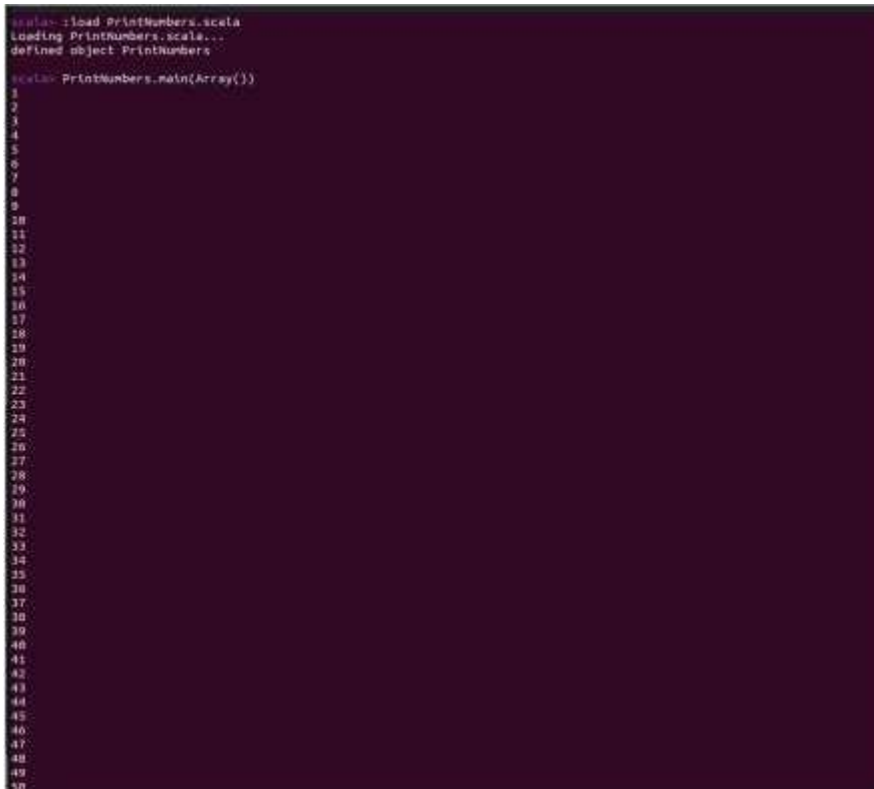
LABORATORY PROGRAM – 8

Write a Scala program to print numbers from 1 to 100 using for loop.

Code

```
object PrintNumbers {  
    def main(args: Array[String]): Unit = {  
  
        for (i <- 1 to 100) {  
            println(i)  
        }  
    }  
}
```

COMMAND WITH OUTPUT



The screenshot shows a Scala REPL session. The first command is `scala> :load PrintNumbers.scala`, which loads the file. The output shows `Loading PrintNumbers.scala...` and `defined object PrintNumbers`. The second command is `scala> PrintNumbers.main(Array())`, which runs the program. The output is a list of numbers from 1 to 100, printed one per line.

```
scala> :load PrintNumbers.scala  
Loading PrintNumbers.scala...  
defined object PrintNumbers  
  
scala> PrintNumbers.main(Array())  
1  
2  
3  
4  
5  
6  
7  
8  
9  
10  
11  
12  
13  
14  
15  
16  
17  
18  
19  
20  
21  
22  
23  
24  
25  
26  
27  
28  
29  
30  
31  
32  
33  
34  
35  
36  
37  
38  
39  
40  
41  
42  
43  
44  
45  
46  
47  
48  
49  
50
```

LABORATORY PROGRAM – 9

Using RDD and FlatMap count how many times each word appears in a file and write out a list of words whose count is strictly greater than 4 using Spark.

Code

```
from pyspark import SparkContext

# Create Spark Context
sc = SparkContext("local", "WordCountRDD")

# Load text file
# Replace 'input.txt' with your file path
lines = sc.textFile("input.txt")

# Split lines into words using flatMap
words = lines.flatMap(lambda line: line.split())

# Map each word to (word, 1)
word_pairs = words.map(lambda word: (word.lower(), 1))

# Count occurrences of each word
word_counts = word_pairs.reduceByKey(lambda a, b: a + b)

# Filter words with count strictly greater than 4
filtered_words = word_counts.filter(lambda x: x[1] > 4)

# Collect and print results
results = filtered_words.collect()

for word, count in results:
    print(word, count)

# Optional: save output to file
filtered_words.saveAsTextFile("output")

# Stop Spark Context
sc.stop()
```


COMMAND WITH OUTPUT

```
bmwecsd@bmwecsd-HP-Pro-Tower-480-G3-PC1-Desktop-PC:~$ spark-shell
25/05/20 10:34:37 WARN Utils: Your hostname, bmwecsd-HP-Pro-Tower-480-G3-PC1-Desktop-PC resolves to a loopback address: 127.0.0.1; using 10.124.3.102 instead (on interface eno1)
25/05/20 10:34:37 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For Spark, use setLogLevel(newLevel).
25/05/20 10:34:49 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Spark context Web UI available at http://10.124.3.102:4040
Spark context available as 'sc' (master = local[*], app id = local-1747730090131).
Spark session available as 'spark'.
Welcome to
      _
     / \_
    /  _ \
   /  _ \
  /  _ \
 /  _ \
/_  _ \
\__\_\
version 3.3.4

Using Scala version 2.12.10 (OpenJDK 64-Bit Server VM, Java 11.0.10)
Type in expressions to have them evaluated.
Type :help for more information.
```

```
scala> :load FilterWordCount.scala
Loading FilterWordCount.scala...
import org.apache.spark.sql.SparkSession
defined object FilterWordCount

scala> FilterWordCount.main(Array("file"))
25/05/20 11:45:23 WARN SparkSession: Using an existing Spark session; only runtime SQL configurations will take effect.
bright -> 1
the -> 11

scala> 
```

LABORATORY PROGRAM – 10

Write a simple streaming program in Spark to receive text data streams on a particular port, perform basic text cleaning (like white space removal, stop words removal, lemmatization, etc.), and print the cleaned text on the screen. (Open Ended Question).

Code

```
# Install NLTK and download required data (run once)
!pip install nltk

import nltk
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')

from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lower, regexp_replace, split, explode, udf
from pyspark.sql.types import ArrayType, StringType
from pyspark.ml.feature import StopWordsRemover
from nltk.stem import WordNetLemmatizer

# Initialize SparkSession
spark = SparkSession.builder.appName("TextProcessing").getOrCreate()

# Define your input lines
lines = [
    "Hello, I hate you.",
    "I hate that I love you.",
    "Don't want to, but I can't put",
    "nobody else above you."
]

# Create DataFrame from lines
df = spark.createDataFrame(lines, "string").toDF("value")

# Step 1: Lowercase and remove punctuation
df_clean = df.select(regexp_replace(lower(col("value")), "[^a-zA-Z\\s]", "").alias("cleaned"))

# Step 2: Tokenize the cleaned text
df_tokens = df_clean.select(split(col("cleaned"), "\\s+").alias("tokens"))

# Step 3: Remove stop words
remover = StopWordsRemover(inputCol="tokens", outputCol="filtered")
df_filtered = remover.transform(df_tokens)

# Step 4: Lemmatization using NLTK WordNetLemmatizer with UDF
lemmatizer = WordNetLemmatizer()

def lemmatize_words(words):
    return [lemmatizer.lemmatize(word) for word in words]

lemmatize_udf = udf(lemmatize_words,
    ArrayType(StringType()))

df_lemmatized = df_filtered.withColumn("lemmatized", lemmatize_udf(col("filtered")))
```

COMMAND WITH OUTPUT

```
Requirement already satisfied: nltk in /usr/local/lib/python3.11/dist-packages (3.9.1)
Requirement already satisfied: click in /usr/local/lib/python3.11/dist-packages (from nltk) (8.2.0)
Requirement already satisfied: joblib in /usr/local/lib/python3.11/dist-packages (from nltk) (1.5.0)
Requirement already satisfied: regex<=2021.8.3 in /usr/local/lib/python3.11/dist-packages (from nltk) (2024.11.6)
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from nltk) (4.67.1)
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
+-----+
|word |
+-----+
|hello |
|hate |
|hate |
|love |
|dont |
|want |
|cant |
|put |
|nobody|
|else |
+-----+
```